FEVER: a plain explanation

Federated Eviction and Volume-based Enforcement against overflow Rules. Notes for the supervisor meeting, 7 November 2026 (updated: features, real data and scaling plan).

1. The problem: low-rate flow-table overflow (LOFT)

In software-defined networking (SDN), a switch does not decide where packets go by itself. It keeps a **flow table**, a list of rules, one rule per connection. When a packet arrives that matches no rule, the switch asks the **controller** (this request is called a **Packet-In**), the controller decides, and it installs a new rule. In our setup a table holds **3,500 rules**, and a rule is deleted after **10 s without any packet** (the idle timeout).

A LOFT attacker opens many fake connections, so the switch creates many rules for it. On each fake connection it sends one tiny packet just before the 10 s timeout. Each fake rule therefore never expires, and it costs the attacker almost nothing. Slowly the table fills with attack rules. When it is full, a normal user's new connection finds no space: it is refused or delayed. Because the attacker sends only a few small packets per second, ordinary DDoS alarms, which look for high traffic rates, never fire.

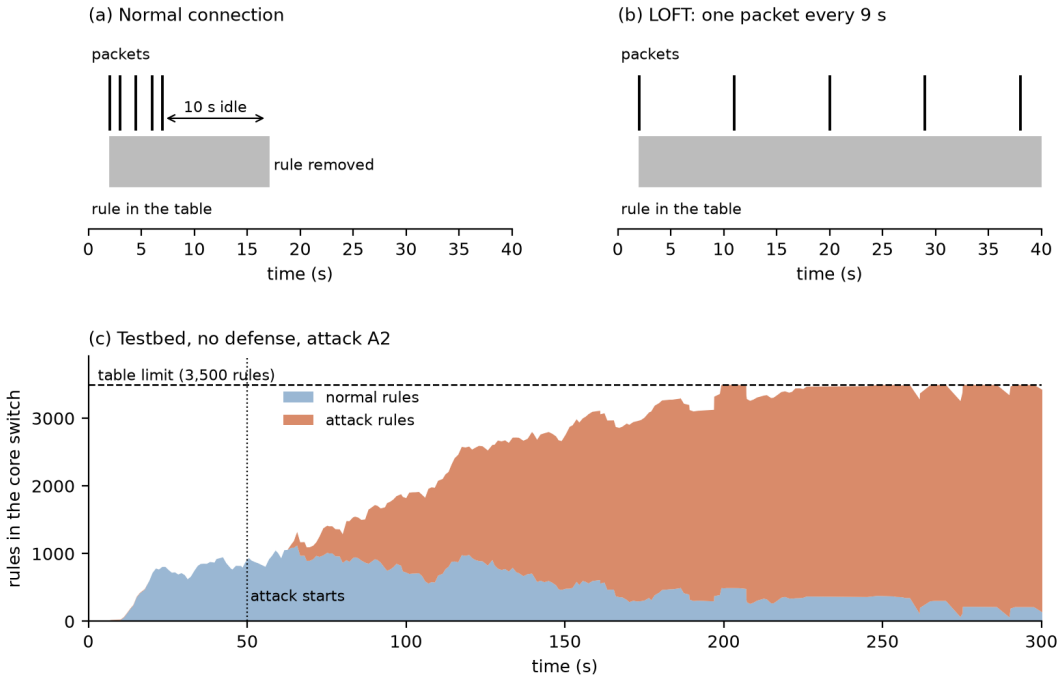


Fig. 1. (a) A normal rule is removed 10 s after its last packet. (b) A LOFT rule receives one packet just before the timeout, so it stays. (c) Our testbed with no defense: attack rules (orange) push out normal rules (blue) until the table is full.

We test three versions of the attack on the testbed:

Table 1. The attacks used on the testbed.

Attack	What it does	Why we use it
A2 (standard)	100 new rules per burst, a burst every 5.5-9 s, one 64-byte packet per rule, from its own address	The attack all related papers test
A4 (disguised)	Same pace, but 1-3 packets per refresh with normal-looking sizes, from a pool of addresses	Imitates normal traffic, like rDefender's advanced attack
E9P (defense-aware)	Refreshes every 2-3 s with normal-looking packets; chosen by a search as the hardest attack against our defenses' settings	Similar in spirit to ALFO's adaptive attack

2. What earlier defenses miss

The related papers (SFTO-Guard, FTMaster, FTODefender, FTOP, DMS-BTT, FloRa, FTShield, rDefender, ALFO-Guard) mostly train a machine-learning model that looks at each rule and decides whether it belongs to an attacker. They then delete the attack rules, and some also block the attacker. They work well on the attack they were trained on. Five things are missing:

Table 2. Gaps in previous work.

Gap	In plain words
1. Each controller learns alone	A controller only knows the attacks it has seen itself. If a neighbouring network saw a new attack, this controller still cannot recognise it.
2. Tested on the attacks they trained on	Papers report about 99 % accuracy, but on the same kind of attack used for training. On attacks seen elsewhere the same detectors fall to about 80 %, and on a new attack to 9-25 % (Section 6.1).
3. Disguised attacks hardly tested	Only rDefender and ALFO test attackers who imitate normal traffic. A detector that judges how packets look fails against them.
4. Harm to users not measured	Most papers report how full the table is, not how many normal users were refused or how overloaded the controller became.
5. Same data for training and testing	Trace-based papers often train and test on the same capture, which makes results look better than on new traffic.

3. Our idea and contributions

The idea in one sentence. Let the controllers share what they have learned (their trained models, never their traffic), and add a simple rule that does not depend on machine learning: a host that holds far more rules than anyone else is blocked, whatever its packets look like.

1 **Federated detection across controllers.** Each controller trains a model on its own attacks and shares the model. Every controller can then recognise attacks only the others have seen. No related paper does this.

2 **Evaluation by exposure.** We test every detector on three groups: attacks seen in training, attacks seen only by another controller, and an attack nobody saw. This shows how detectors behave on attacks they did not train on.

3 **Two signals that cover each other.** The detector judges *what a rule looks like*; the volume rule judges *how many rules a host holds*. When an attacker imitates normal traffic, the detector fails, but the attacker still has to hold many rules to fill the table, so the volume rule catches it.

4 **A cost for the attacker.** To stay under the volume limit (about 800 rules per host), an attacker needs at least 4 colluding hosts on one switch to fill the table (to be tested).

5 **A careful evaluation.** A real testbed, a clean split of the traffic into training and test parts, hypotheses fixed before the test runs, and measures of harm to users.

What is not new, and should not be claimed: deleting suspicious rules, blocking the attacking host (also permanently), a fair share of the table per port, and tree-based classifiers. These all appear in earlier papers. The name FEVER stands for **F**ederated **E**viction and **V**olume-based **E**nforcement against overflow **R**ules.

4. How FEVER works

FEVER has two halves. **Offline**, the controllers build a shared detector together. **Online**, every controller runs the same short loop once per second on each of its switches.

4.1 Offline: building the shared detector

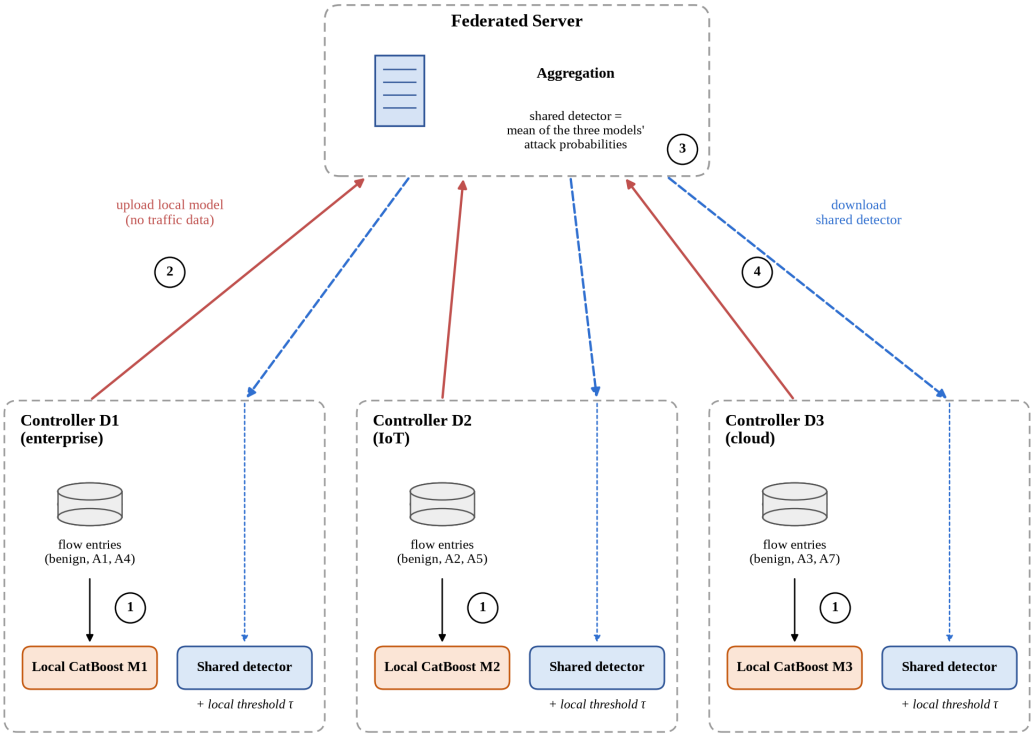


Fig. 2. Federated training: (1) each controller trains locally, (2) uploads its model, (3) the models are combined, (4) every controller downloads the shared detector.

1 **Three controllers with different experience.** D1 has seen attacks A1 and A4, D2 has seen A2 and A5, D3 has seen A3 and A7. Nobody has seen A6 or E9P; we keep those to test new attacks.

2 **Each trains a CatBoost model** (a fast tree-based classifier) on its own normal traffic and attacks. Each rule is described by 17 numbers (features): 11 about the connection (packet rate, gaps between packets, packet sizes), 3 about the host that created it, and 3 about the table (how full it is, how fast it changes). Section 4.3 lists them.

3 **Models are shared, data is not.** The shared detector runs all three models on a rule and takes the average of their attack probabilities. If any one controller has seen this kind of attack, its model gives a high score and lifts the average. (This is model sharing; it is not FedAvg, which averages the models' weights.)

4 **Each controller sets its own limits from its own normal traffic:** the score threshold $\tau = 0.620$, chosen so that only 1 in 1,000 normal rules scores above it, and the volume limit X of about 800 rules, 1.5 times the busiest normal host ever seen. On the testbed both come from the training part of the traffic only.

4.2 Online: the loop every second

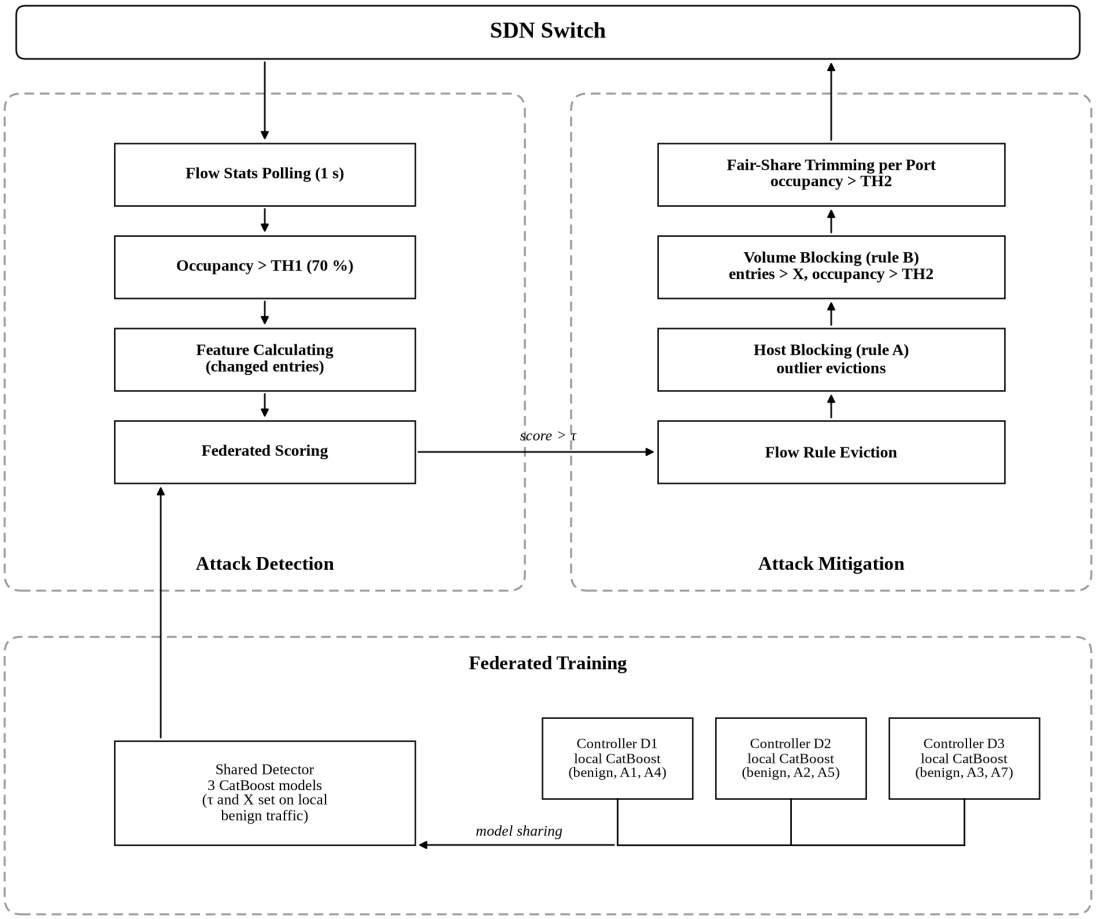


Fig. 3. FEVER architecture: attack detection (left) and attack mitigation (right) on each controller; the shared detector comes from federated training (bottom).

Read the left column from top to bottom, then the right column from bottom to top:

Table 3. The online loop, step by step, and why each step is there.

Step	What happens	Why
1 Poll	Read every rule's counters (packets, bytes, age, source host and port) and how full the table is.	Everything below needs these numbers.
2 Gate (TH1)	Continue only if the table is at least 70 % full.	Below 70 % there is no danger; this saves controller time.
3 Features	Compute the 17 features, only for rules whose counters changed.	Scoring unchanged rules again is wasted work.
4 Score	The three shared models score each rule.	Recognise attack rules, including attacks only other controllers saw.
5 Evict	Delete every rule scoring at least τ.	Frees space at once.
6 Block rule A	Block a host whose evictions in 10 s are at least 50, at least 3 times the median host's (counted as at least 10), and at least 20 % of its own rules.	Otherwise the attacker simply creates new rules. The floors keep a busy normal host from being blocked over a few false alarms.
7 Block rule B (volume)	If the table is at least 90 % full (TH2): block a host holding more than X rules and at least 3 times the median host's.	Catches disguised attackers the detector cannot see: whatever they look like, they must hold many rules.
8 Fair share	If the table is still at least 90 % full, each port keeps a weighted fair share; the highest-scoring rules above it are removed.	A safety net for anything both rules miss.

Blocking means a drop rule at the host's switch port: its packets no longer reach the table or the controller. In the current version (v6) the block is **permanent**. The earlier version (v5) released the host after 60 s, and under A2 the attackers came back and filled the table again (Section 6.3).

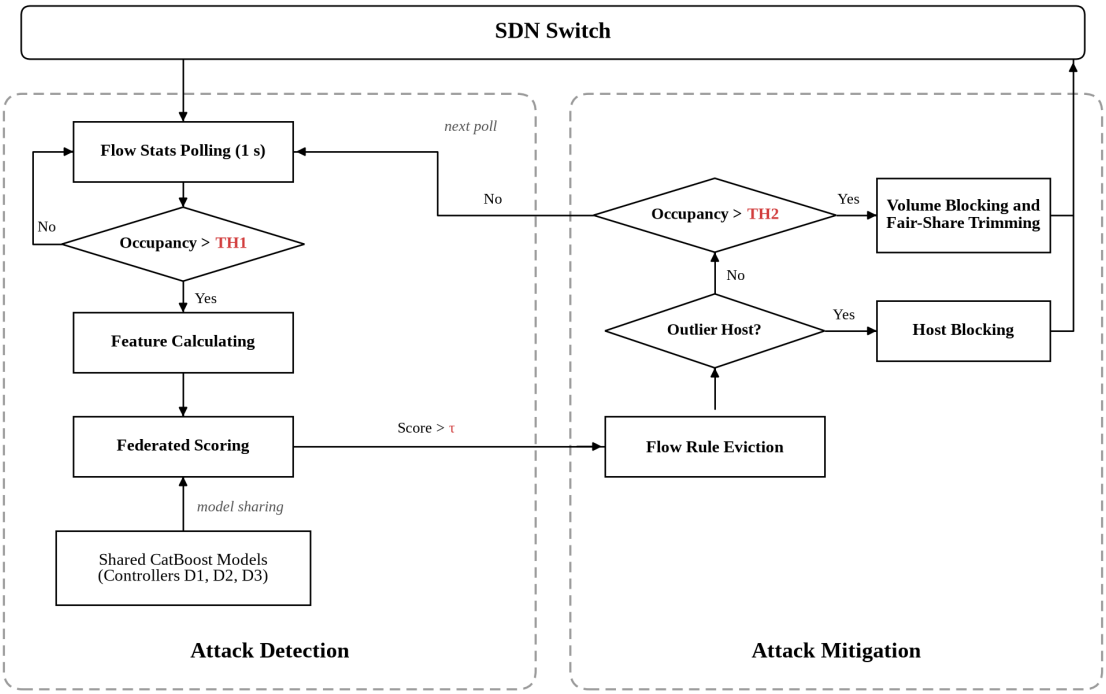


Fig. 4. The same loop as a flowchart (TH1 = 70 %, TH2 = 90 %).

4.3 The 17 features used for scoring

Every rule in the table is described by 17 numbers. All of them come from what any OpenFlow 1.3 controller already sees: each rule's packet and byte counters, its age and its source address, plus the controller's own memory of earlier polls. No packet contents are inspected and no extra hardware is needed. The features sit at three levels: the rule itself, the host that created it, and the whole table (loftbench/features.py).

Table 4. The 17 features, their level, and where similar features appear.

#	Feature	Level	In plain words	Similar feature in
1	log_pkts	Rule	How many packets the rule has matched	all ML papers
2	log_bytes	Rule	How many bytes it has matched	all ML papers
3	log_dur	Rule	How long the rule has existed	all ML papers
4	mean_size	Rule	Average packet size	FTMaster, FTOP, FTODefender, DMS-BTT
5	log_rate	Rule	Packets per second	FTOP, FloRa
6	log_pit	Rule	Average time between packets	DMS-BTT (PIT), FTMaster, FTODefender
7	log_dpkts	Rule	Packets since the last poll (1 s ago)	ours
8	idle_polls	Rule	Seconds since the rule last received a packet	ours
9	active_ratio	Rule	Share of polls in which the rule received packets	ours
10	gap_cv	Rule	How regular the gaps between its packets are	FloRa (arrival pattern)
11	gap_mean	Rule	Average gap between bursts of activity	FloRa (arrival pattern)
12	src_entries	Host	How many rules the same host holds	ours, per rule
13	src_new	Host	How many rules that host created in the last 10 s	ours, per rule
14	src_small_frac	Host	Share of that host's rules with at most 3 packets	ours
15	occupancy	Table	How full the table is	FTMaster, DMS-BTT (state)
16	install_rate	Table	New rules per second in the last 5 s	ours
17	table_apit	Table	Average time between packets over all rules	DMS-BTT (APIT)

Why three levels. One rule on its own often looks harmless. Rule-level features say how the rule behaves; host-level features say what its owner is doing (a host creating hundreds of tiny rules is suspicious); table-level features say whether the table is under pressure. A LOFT rule typically has one small packet, then a long silence of about 7 s (features 1, 4, 8, 9), comes from a host with many small new rules (12-14), and appears while the table is filling quickly (15, 16).

Is there novelty in the features? Only a little, and the features should not be presented as a main contribution. Most rule-level features (1-6, 10-11) and two table-level ones (15, 17) appear in earlier papers. What is different:

- **Host context attached to every rule** (12-14). FTMaster and FTODefender measure the sources only for the whole table (the entropy of source addresses, new sources), to decide *whether* an attack is happening. FEVER gives each rule its own host's numbers, so the model can tell *which* rules belong to an attacking host.
- **Keep-alive pattern from the poll history** (7-9): how long a rule has been silent and how often it is active. LOFT rules are kept alive by one packet just before the timeout, and these features describe that directly. We have not seen them in the related papers we reimplemented.
- **One model sees all three levels at once.** DMS-BTT, FTMaster and FTOP work in two separate stages (a table-level check, then a rule-level classifier). FEVER scores each rule with its rule, host and table features together.

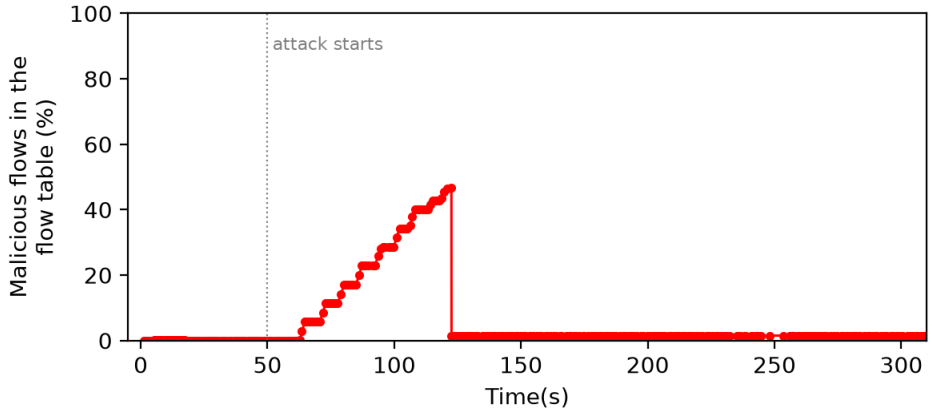
- **Features that transfer between networks**, which federation needs: every controller must compute exactly the same features, and a model trained in one network must work in another. Counts are log-scaled and table features are relative to the table size, so the shared models do not depend on one network's table size or traffic volume.

Feature 12 (rules per host) is the same quantity the volume rule uses. The difference: in the detector it is one input among 17, weighed by the model; in the volume rule it decides on its own. On the testbed the model still scores A4 and E9P rules low, because their other features look normal, which is why the separate volume rule is needed.

4.4 What happens during an attack

Under A2 (standard). The attack rules accumulate until the table passes 70 %. The detector recognises them (D2 trained on A2), so they are evicted. Each attacker's evictions soon pass 50 in 10 s and stand far above everyone else's, so block rule A blocks both attackers, about 70 s after the attack starts. Their rules disappear and, because the block is permanent, they never return.

Under A4 and E9P (disguised). The attack rules look normal, so the detector scores them low and evicts few. The table keeps filling. Once it passes 90 %, each attacker holds far more than X rules, far more than any normal host, and block rule B blocks it, about 70-125 s after the attack starts.



Percentage of malicious flows in the flow table with FEVER deployed (A2 attack).

Fig. 5. Share of attack rules in the table with FEVER under A2 (one testbed run): it rises until both attackers are blocked, then stays near 0 %.

5. How we tested

5.1 Detection benchmark (offline)

Every detector, ours and the published ones, is trained and scored on exactly the same data from three simulated networks (D1-D3), at a false-alarm rate of 0.1 %. We report each detector on attacks seen in training, attacks seen only by another controller, and an attack nobody saw.

5.2 Testbed (online)

- **Software:** Mininet (a virtual network on one laptop), Open vSwitch, the Ryu controller, OpenFlow 1.3. Flow table 3,500 rules, idle timeout 10 s, the same as the Tang et al. papers.
- **Topology:** a core switch connected to the server, two access switches, each with two normal users and one attacker (Fig. 6 and Fig. 7).
- **Normal traffic:** a real traffic capture (bigFlows) replayed. Its first 40 % (T1A) is used only for training models and setting limits, for every method; its last 60 % (T1B) only for testing.
- **Each run** lasts 300 s; the attack starts at 50 s. Five runs per case (seeds 4140-4144), one run at a time on an otherwise idle laptop.
- **Preregistration:** the hypotheses and tests were written down and committed before the test runs.
- **Methods compared:** no defense, FTMaster, DMS-BTT, rDefender, FEVER (and earlier EVICT, SIFT, TableGuard, FTOP and our older versions). All were reimplemented and retrained on T1A. Our DMS-BTT has a 60 s blocklist that the original paper does not have, so it is stronger than published.

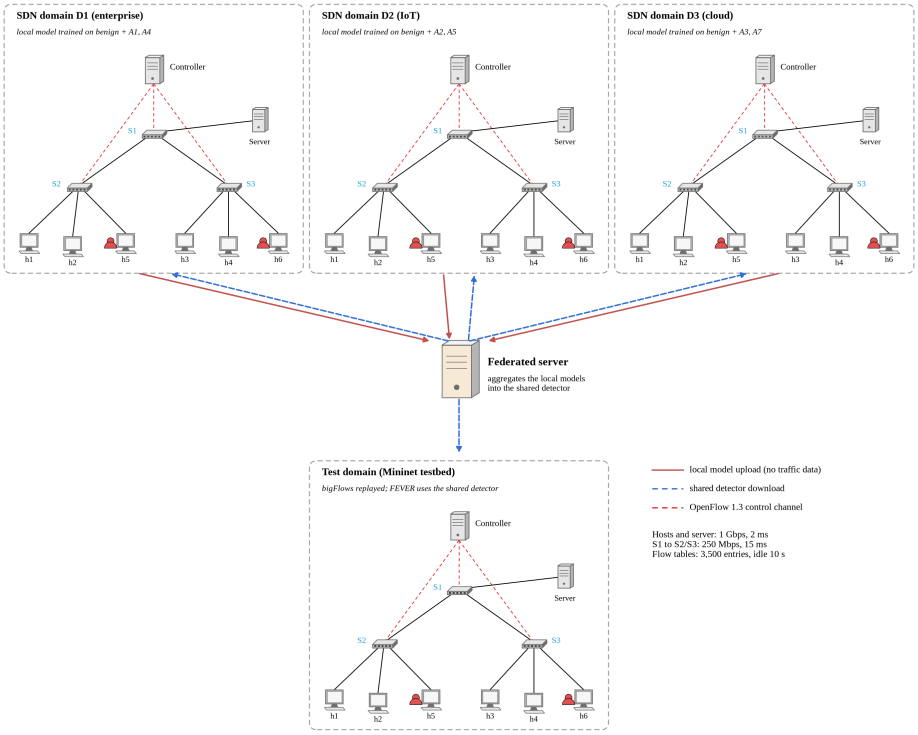


Fig. 6. Experiment topology: three domains train and share models; the shared detector is used in the test domain (the testbed).

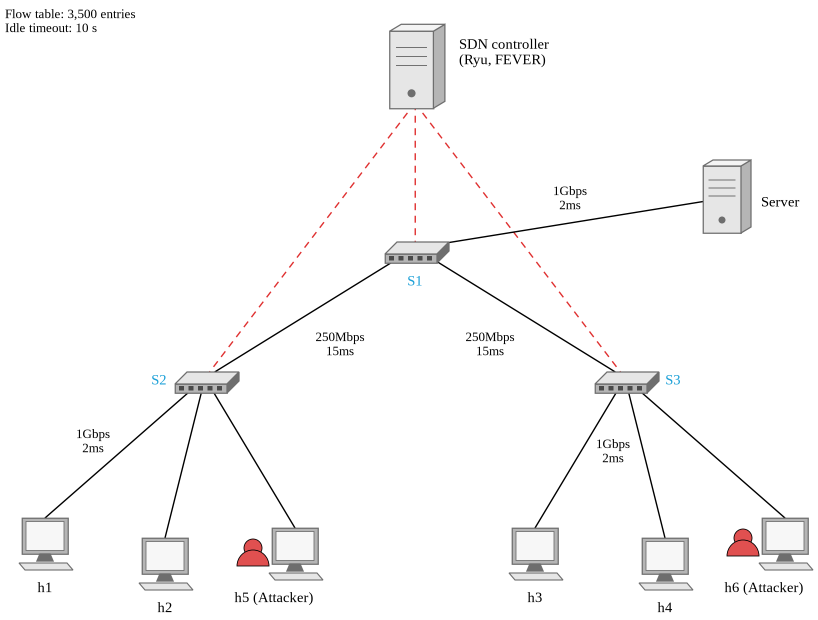


Fig. 7. Network topology of the testbed.

Table 5. What we measure.

Measure	Meaning	Better
Benign flows harmed	Normal connections refused or failed	lower; the most important
Attacker share	Part of the core table held by attack rules	lower
Lost first packets	Normal Packet-In requests the overloaded controller drops	lower
New connections succeeding	A probe keeps opening new connections; how many work	higher
Decision time	Time the controller needs per poll	lower
Innocent hosts blocked	Normal users wrongly cut off	must be 0

5.3 What is real and what is generated

Table 6. Real and generated parts of the experiments.

Part	Real or generated	Note
Switch and controller software (Open vSwitch, Ryu, OpenFlow 1.3)	Real	The software used in practice and by the Tang et al. papers; runs in Mininet on one laptop
Normal traffic on the testbed	Real	bigFlows, a real network capture (368 MB), replayed; split by time into training (40 %) and test (60 %) parts
Attack traffic	Generated	No public LOFT dataset exists; every related paper generates its attacks too
The three federated training networks (D1-D3)	Simulated	Built from traffic models in our simulator; to be replaced by real traces (Section 7.2)
Federation	Offline	Models are trained and shared before the experiment, then loaded by the controller; to be run live between controllers (Section 7.2)

6. Results

6.1 Detection: sharing models helps on attacks a controller never saw

Detector	Attacks seen in training				Attacks seen only by another controller				New attack seen by no controller			
	Acc.	**Prec.**	**Rec.**	**F1**	**Acc.**	**Prec.**	**Rec.**	**F1**	**Acc.**	**Prec.**	**Rec.**	**F1**
FTOP	96.94	99.79	92.02	95.02	92.59	84.33	79.43	81.16	81.18	35.50	9.49	14.74
FTMaster	97.07	99.79	92.37	95.30	93.34	84.41	81.29	82.20	82.37	35.64	15.00	20.65
FloRa	99.01	**99.82**	97.50	98.62	93.65	84.30	82.19	82.68	84.51	35.48	24.90	28.46
DMS-BTT	97.03	99.78	92.27	95.22	93.45	84.29	81.62	82.38	82.21	35.36	14.28	19.94
FEVER	**99.61**	99.75	**99.19**	**99.46**	**99.43**	**99.75**	**98.60**	**99.15**	**89.48**	**98.99**	**50.98**	**66.81**

Fig. 8. Detection results at a 0.1 % false-alarm rate (same data for all).

- On attacks used in training, every detector is good (F1 95-99 %).
- On attacks seen only by another controller, FEVER keeps 98.6 % recall; the published detectors fall to 79-82 %. This is the direct benefit of sharing models.
- On the attack nobody saw, FEVER catches 51 %, the others 9-25 %.

6.2 Testbed: FEVER against the published methods

Table 7. Testbed results, mean of five runs per attack (A2, A4, E9P); FEVER with permanent blocking (v6).

	FEVER	**DMS-BTT**	**FTMaster**	**No defense**
Benign flows harmed (all attacks)	**0.01 %**	1.71 %	4.80 %	10.13 %
Benign flows harmed under A2	**0.01 %**	0.02 %	0.01 %	-
Benign flows harmed under A4	**0.02 %**	2.09 %	4.16 %	-
Benign flows harmed under E9P	**0.00 %**	3.01 %	10.22 %	-
Attacker share of the table	**11.9 %**	41.9 %	45.5 %	62.1 %
Lost first packets	**9.6 %**	32.6 %	27.8 %	-
New connections succeeding	**37.3 %**	23.4 %	23.7 %	-
Attackers blocked	30 of 30	0	0	-
Innocent hosts blocked	0	0	0	-

FEVER is best or tied on every measure under every attack. Under A2 it ties the two published methods on harm and is better on everything else; under the disguised attacks it is far better.

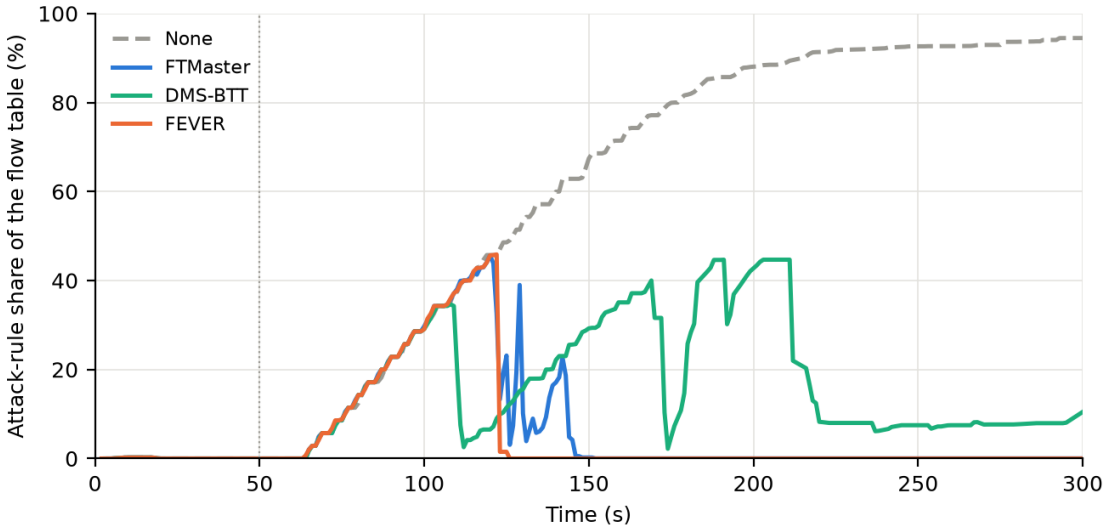


Fig. 9. Attacker share of the table over time under A2 (median of five runs). FEVER drops it to 0 % at about 120 s and keeps it there.

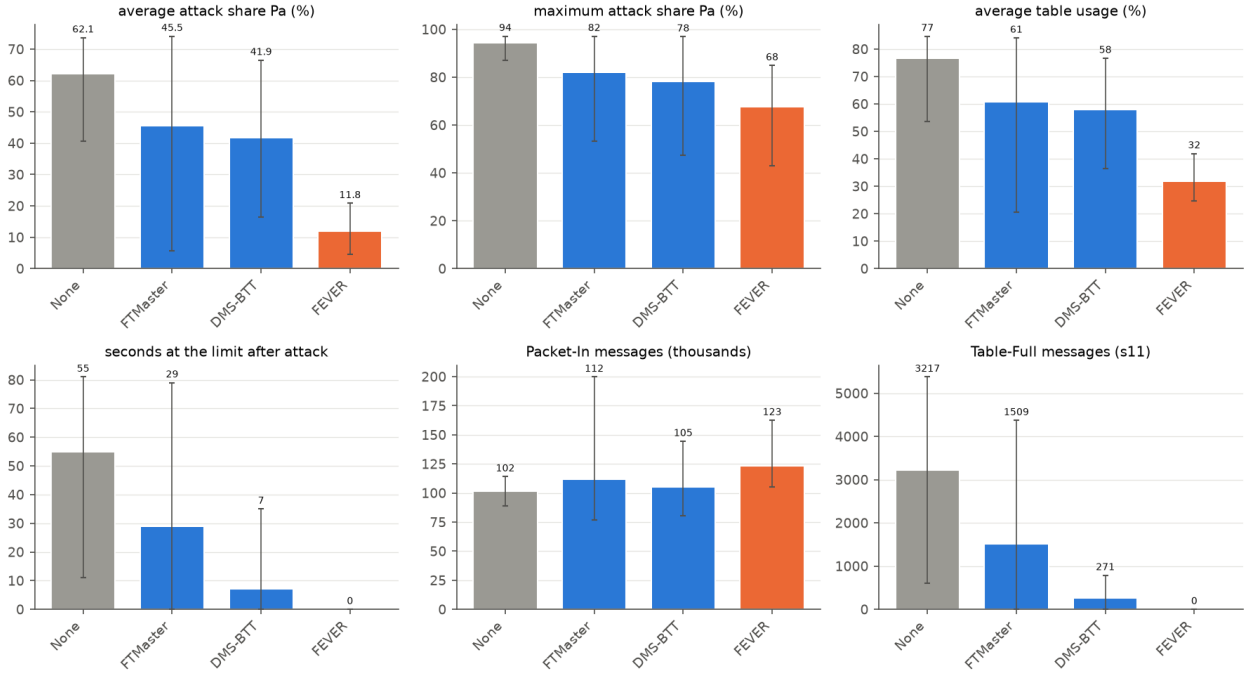


Fig. 10. Comparison under all three attacks (bar: mean; whisker: range).

6.3 Why permanent blocking (v5 against v6)

In the preregistered campaign (120 runs), FEVER blocked a host for only 60 s. It already won under A4 and E9P, but lost under A2 (harm 1.05 % against 0.01-0.02 %): when the block expired, the attackers came back and filled the table again. Version v6 changes only this one setting: a blocked host stays blocked, as in FTMaster, FTODefender and FloRa. The change and its acceptance test were written down before running it.

Table 8. FEVER v5 (60 s block) and v6 (permanent block) on the same runs.

	v6	**v5**
Benign flows harmed under A2	0.01 %	1.05 %
Benign flows harmed, all attacks	0.01 %	0.49 %
Attacker share	11.9 %	21.9 %
Lost first packets	9.6 %	19.8 %
Innocent hosts blocked	0	0

6.4 Which part does what (ablation)

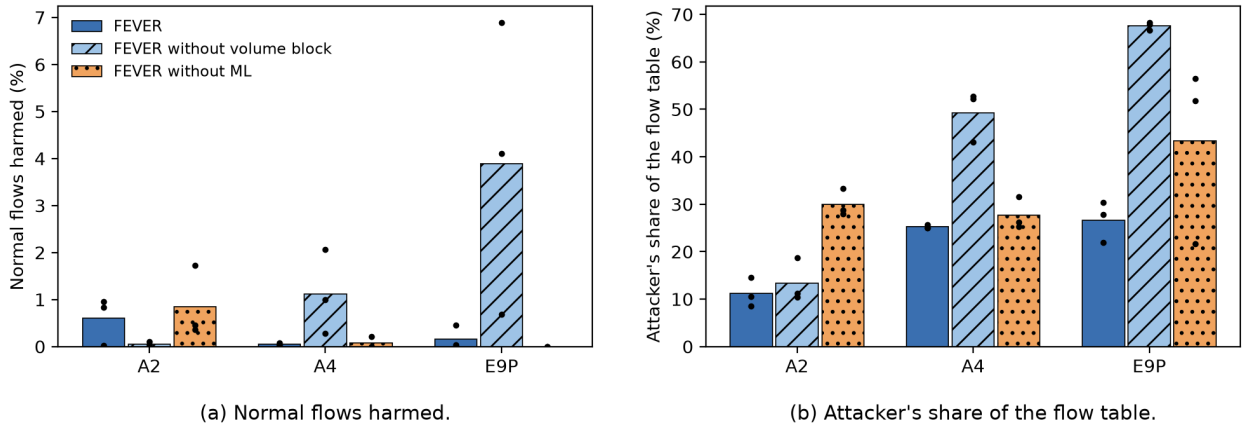


Fig. 11. Ablation (60 s-block version, three runs per attack): FEVER, FEVER without the volume rule, FEVER without the detector, and rDefender.

- **Without the volume rule**, harm under A4 and E9P rises to 1.11 % and 3.89 % and the attackers are never blocked: the volume rule is what stops disguised attacks.
- **Without the detector**, the A2 attackers are blocked at about 130 s instead of 71 s and keep 30 % of the table instead of 11 %: the detector stops standard attacks faster.
- So the two signals cover each other's blind spot.

6.5 Cost

Method	**CPU usage (%)**			**Memory (MB)**		**Time per decision (ms)**
	No attack	**Increase over no defense**	**Under attack**	**Peak**	**Increase over no defense**	
None	48.2	-	92.9	216	-	30
FTMaster	48.6	+0.4	91.2	219	+2	72
DMS-BTT	52.4	+4.3	94.6	243	+27	65
FEVER	49.2	+1.1	88.9	291	+75	11

Fig. 12. Controller overhead on the testbed.

FEVER decides in about 11 ms per poll (DMS-BTT 65 ms, FTMaster 72 ms) and adds about 1 % CPU without an attack. It uses the most memory (+75 MB peak), because three models are loaded.

7. What we still need to do

7.1 Open points

Table 9. Open points.

Weak point	What we will do
The v6 result is not yet a clean test: v6 was designed after seeing the A2 result and ran on the same seeds.	One final campaign on new seeds (4160-4164) with every method, about 160 runs (14 h).
FTODefender, FTOP and FloRa are not yet in the testbed comparison.	FTODefender is implemented; train it on T1A, then include all three in the final campaign.
A wrongly blocked host is cut off for good. The busiest normal host holds about 580 rules against a limit of about 800.	Test a busy but legitimate host (for example a NAT gateway). Later: release a host with a cap on its rules.
An attacker could spread over many hosts to stay under the volume limit.	Adaptive-attacker test, and write up the cost (at least 4 hosts per switch).
On the testbed, the shared detector does not catch disguised attacks; the volume rule does.	Present federation (offline evidence) and the volume rule (testbed evidence) as separate contributions.
New users still suffer under every defense (37 % of new connections succeed).	The single controller is overloaded; test a Packet-In rate limit or a separate controller machine.
Narrow scope: one laptop, one trace, one topology, five seeds, 300 s runs.	A second trace as a separate test network; more seeds; state it in the limitations.

7.2 Plan to make it scalable and use more real data

Table 10. Scaling plan.

Step	What we will do	What it shows
1 More real data	Add univ2 (the university data-centre trace used by FTOP, already downloaded) as a second test network. Train the three federated controllers on different real traces (bigFlows, univ2, CAIDA or MAWI backbone traces) instead of simulated networks.	That the results hold on real traffic from several networks, not one capture
2 Live federation	Run three Ryu controllers, each with its own Mininet network, that train locally and exchange models over the network during the experiment. Measure the traffic this costs (each model is about 1 MB) and grow from 3 to 5 and 10 controllers.	That federation works as a running system and scales with the number of controllers
3 Larger networks	Fat-tree and larger tree topologies with more switches and hosts; table sizes from 1,000 to 10,000 rules; other idle timeouts; higher traffic rates.	That FEVER is not tied to one small topology or table size
4 Better hardware	Move from the laptop to a lab server or cloud machines, with the controller on its own machine; if available, a physical OpenFlow switch or Raspberry Pis (as FloRa used).	Results without one overloaded laptop; closer to a real deployment
5 Stronger tests	Final campaign on new seeds with every method; longer runs; an attacker spread over many hosts; a busy but legitimate host.	A clean, confirmatory result and the limits of the volume rule

Before the NetSoft deadline (late January 2027), steps 5, 1 and 2 are feasible; steps 3 and 4 depend on lab resources and fit the thesis after the paper.

8. Questions to expect

Table 11. Likely questions and short answers.

Question	Answer
Is this really federated learning?	It is federated model sharing: models are trained locally and shared, and their predictions are averaged. Not FedAvg. No traffic leaves a network.
Why not use only the volume rule?	Without the detector, standard attacks are blocked about 60 s later and keep three times more of the table (Section 6.4).
Why did v5 lose under A2?	Its 60 s block expired, the attackers returned and refilled the table. Permanent blocking fixed it.
Is permanent blocking safe?	No innocent host was blocked in 40 runs, but the margin is small; the busy-host test will check it.
Why are DMS-BTT and FTMaster so good under A2?	They were trained on A2 traffic from the same network and delete attack rules every second. Our DMS-BTT also has an added 60 s blocklist.
Is the comparison fair?	Every method is retrained on the same training data and run on the same testbed and seeds. The numbers are not comparable with those printed in the papers, because each paper uses its own setup.
What is new compared with rDefender?	rDefender limits each port's share but never finds or blocks the attacker. FEVER identifies the attacking host, by detector or by volume, and removes it.

9. Words used

Table 12. Glossary.

Word	Meaning
Flow table / rule	The switch's list of forwarding instructions, one per connection
Idle timeout	A rule is deleted after this long (10 s) without packets
Controller / Packet-In	The program that installs rules; the switch's request to it for a new rule
Evict	Delete a rule from the table
Block	Drop all packets of a host at its switch port
τ (tau)	The score above which a rule is treated as an attack rule
X	The volume limit: the number of rules above which a host is suspicious
TH1 / TH2	Table fullness at which scoring starts (70 %) and the volume rule acts (90 %)
T1A / T1B	Training part (first 40 %) and test part (last 60 %) of the traffic capture
Seed	A number fixing the random choices of one run, so it can be repeated
Recall / F1	Share of attack rules caught / a balance of recall and precision